Pergamon

Computers Elect. Engng Vol. 20, No. 2, pp. 151–162, 1994
Copyright © 1994 Elsevier Science Ltd
Printed in Great Britain. All rights reserved
0045-7906/94 $6.00 + 0.00

A RULE-BASED EXPERT SYSTEM APPROACH TO CLASS SCHEDULING

L. GUYETTE, K. HAMIDIAN and J. O. TUAZON
Department of Electrical Engineering, California State University, Fullerton, Fullerton, CA 92634, U.S.A.

(Accepted for publication 15 June 1993)

Abstract—Class scheduling is the unenviable task in which a team of scheduling experts must assign faculty members their requested courses based on the university and departmental guidelines. The complexity of this task is compounded by introducing the various necessary faculty and course parameters. Although there have been recent attempts to create the class scheduling process via conventional programming, this form of software is poorly suited to emulate the essential human decision-making process. However, a rule-based expert system approach can provide the practical solution. This paper is concerned with originating the formulation and implementation necessary to achieve an optimal class scheduling solution.

1. INTRODUCTION

The activity, in which faculty members request and obtain courses that they want to teach, is known as class scheduling. The current class scheduling process requires both a large amount of time and extreme dedication from a human scheduling expert. However, an expert system can provide more reliable and consistent scheduling solutions and in a much shorter period of time than the human expert. Therefore, this paper is concerned with designing and implementing a class scheduling expert system to replace the human expert [1–8].

1.1. Problem description and significance

The ability to schedule courses to meet the individual needs of each faculty member and the related department is based upon logic that is developed from the human decision-making process. This complex process requires that the scheduling expert examine the essential university guidelines along with the various faculty and class parameters to logically implement course assignments for each faculty member. Several techniques may be used to help the human expert to reduce the overall time necessary to complete the class scheduling operation. However, since the human decision making process could not be emulated with conventional programming approaches, the human expert could not be released to perform other essential tasks.

1.2. Solution to the problem

To eliminate the need to involve a scheduling expert in the class scheduling process, the human decision-making mechanism must be properly emulated. This paper has developed the knowledge and logic necessary for the class scheduling expert system to replace the current human expert.

2. PROJECT FORMULATION AND IMPLEMENTATION

The main objective of this paper is to develop and implement a rule-based expert system for class scheduling. The derived system presents a user-friendly environment while maintaining a powerful rule base. The natural language user interface is comprised of a collection of files that are external to the expert system knowledge base. The system logic or rules of the class scheduling expert system are derived from seven logic specific modules in the expert system knowledge base. Together, these files and modules form the bases for a fully functional class scheduling expert system.

2.1. User interface

For an expert system to be operational, knowledge concerning the specific application must be supplied. If the application knowledge is changed, then the input to the expert system must be modified. The application versatility which is allowed by the modification of data unfortunately creates a burden for the user. The more complex the input data, the greater the expertise the user must have. Therefore, one of the main objectives of this expert system is to create a user-friendly environment. This may include the creation of a natural language interface to simplify the input of data to the expert system. Another technique to generate a user-friendly environment is to reduce the amount of knowledge the user needs to know about operating the expert system. The class scheduling project combines both of these approaches to produce a generalized expert system.

2.1.1. Input/output files. To achieve a user-friendly environment, the class scheduling project uses natural language in a file structured system. As shown in Fig. 1, the files utilized in the class scheduling expert system include a Time File, Faculty File, Class File and Result File. Each file is responsible for the data input, also known as the initial facts of the knowledge base, or the output. The information found in the input files allow the expert system to be custom tailored for a specific application. Please note, each of these files may be modified with the use of a simple word processor.

The Time File provides the expert system with knowledge concerning the available class time slots. This information advises the expert system when classes may be taught in a given week. Each block of information in this file is constructed in the following format:

time-chart [slot_number] [day_of_the_week] [hours]
time-chart display [slot_number] [display_format]

The "time-chart" statement at the beginning of each of these lines indicates to the expert system that time slot information is provided. Since there are various class time blocks during the week, each of these time slots was given a "slot number". Along with this information, the "day of the week" and the "hours" for that day is provided. The line that begins with "time-chart display" contains the same information as the previous line, but modifies this information into a user-friendly format. Since each university may make use of different time slots, the time file was not embedded into the expert system. If it was, then the expert system could not easily be customized for universities that operate on a different time base.

At the beginning of the class scheduling process it was assumed that the department would provide information on its faculty members. This information included their names and their seniority or priority number. Along with this data, each faculty member provided information about his or her class teaching preference, available lecture time and the number of units that he or she wanted to teach. This knowledge is presented in the Faculty File with the following block format:

PROF_PRIORITY: [overall_priority_of_faculty_member]
PROF_NAME: [name_of_faculty_member]
PROF_CHOICE: [class_teaching_preferences]
PROF_TIME: [available_lecture_times]
PROF_UNITS: [preferred_class_load]

Each faculty member will have his or her own respective knowledge block for the expert system to draw from. This knowledge will tailor the expert system to each faculty members' need.

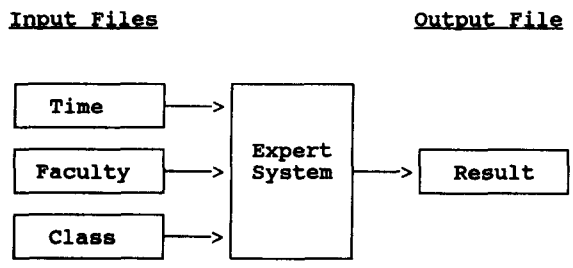


Fig. 1. Input/output file structure.

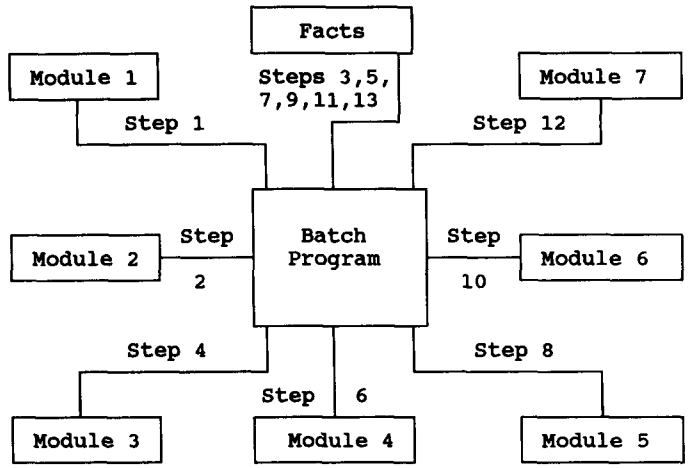


Fig. 2. Batch sequence.

The Class File supplies the expert system with knowledge involving course parameters:

CLASS_NUMBER: [course_number]
CLASS_SEMESTER: [semester_for_course_to_be_taught]
CLASS_TIME: [available_time_period_in_which_class_will_be_taught]
CLASS_ROOM: [selected_room_for_course]
CLASS_SECTIONS: [course_section]
CLASS_UNITS: [number_of_units_for_course]
CLASS_WEIGHT: [weight_of_course]

In the class scheduling project, each course will have a dedicated block of information such as the course number, available lecture times, course section and the number of units for that course.

Once the expert system has generated the solution for the specific application, the results are displayed on the CRT and stored in the Result File. The solution format indicates the faculty member, his or her priority, and the assigned courses:

Professor [name], Priority [priority_number]
class: [course] section: [section] time: [time]
⋮ ⋮ ⋮
class: [course] section: [section] time: [time]

To allow for future reference, this information will remain in the Result File until it is purposely erased.

There are various advantages to this type of file structure. One advantage is that the expert system may be tailored to the needs of universities that may have different course options or time slots. The expert system may also be customized to each faculty members needs. Another advantage is that the files provide a user-friendly environment. A nonexpert can use a simple word processor to modify files, which are easy to understand. And lastly, once the information is stored in this file system, the data may be used again and again. Since the faculty staff and the courses taught will not vary greatly from semester to semester, the information in the files will need only quick and easy modifications between school terms.

2.1.2. Batch file. Under normal operations, the class scheduling expert system is driven by a repetitive sequence of commands to clear the existing rules and facts, load a different set of rules, reset the new rules, load previously created facts, run the rules and save the newly generated facts. However, the class scheduling expert system reduces the amount of knowledge the user needs to know about the expert system environment. This is accomplished with a batch file, which is a collection of top-level commands. The expert system operation is simple. Once in the expert system environment, a simple command will access the class scheduling batch program, which will then control the entire operation of the expert system. The batch program sequence will consist of 13 primary actions, as shown in Fig. 2. For example, step 4 will load module 3 and then step 5 will access the facts.

Table 1. Logic modules

Module No.	Function
1	CRT display
2	Collect knowledge from input files
3	Modify facts
4	Assign courses to faculty
5	Modify facts
6	Exchange classes between faculty members
7	Display and store results

2.2. Logic modules

The class scheduling expert system is composed of an inference engine and a knowledge base. Since only the knowledge base is unique to a given application, the class scheduling project is concerned solely with generating the rules and facts that form that particular knowledge base. The initial facts were located in the input files while the class scheduling rules were broken down into separate modules. Only one module will be loaded into memory at a given time. This approach will reduce the amount of accessible knowledge and thereby limit the memory consumed and the time necessary to search the rule base. Since each module applies a unique set of logic to a collection of facts, the modules are known as "logic modules".

As shown in Table 1, there are seven unique logic modules in the class scheduling expert system. The logic modules operate sequentially. In other words, after module n has completed its operation, then module n + 1 is activated. The process must follow this form since the input for one module is dependent on the previous modules output.

2.2.1. Module 1. Module 1 provides a superficial operation. This module is concerned with displaying to the CRT information concerning the expert system title, the knowledge base engineer's name and acknowledgments. This module also contains a software timing loop to keep the display on the CRT for a predetermined amount of time. At the completion of this module, the class scheduling process will begin.

2.2.2. Module 2. When module 2 is activated, all pertinent class scheduling facts are still contained in the input files. Module 2 collects the information from the files and places the knowledge into working memory.

The initial portion of the Module 2 process is simple. One input file is chosen out of the three possible input files. That file is then opened and the information from the file is gathered word-by-word. After that, the process begins to grow more complex. As each word is taken from the input file, it must be checked to verify that it does not signify the end of the input file or the beginning of a new fact. If the word does not fit either of these description, then it is collected and stored into an existing open fact. If, on the other hand, the word represents the end of the input file, then that file is closed and the next input file is opened. If however, the gathered word represents the beginning of a new fact, then the old fact is closed and a new fact is generated. For example, a fact concerning a faculty member will take the following form:

(faculty PROF_PRIORITY: [priority_number] PROF_
NAME: [name] PROF_CHOICE: [requested_courses]
PROF_TIME: [available_time] PROF_UNITS:
[requested_units])

In this example, every time the expert system "reads" the term faculty from the Faculty File, it starts to generate a fact on the new faculty member. All the words that followed, until the next "faculty" word, will be stored in this fact. This process continues until each faculty member is represented by a unique fact. The entire process continues until all three input files have been read.

As these facts are gathered, the expert system begins to generate facts about the gathered data. For example, by tracking the number of faculty facts that were created, the expert system establishes a separate fact indicating the size of the faculty staff. Therefore, at the completion of Module 2, the expert system has a working knowledge of each faculty member, each class, available time slots, number of faculty members and the number of classes available to be taught.

2.2.3. Module 3. At the completion of logic Module 2, the faculty, class and time facts are stored in the working memory. However, these facts are still a product of a friendly user-interface.

In other words, the input file structures were developed to ease the users ability to interface with the expert system. The unfortunate ramifications of this is that the friendlier the user interface, the more difficult the process for the expert system to decode the information. Module 3 is responsible for transforming the facts from Module 2 into a workable set of facts for the next logic module.

The main process of Module 3 is to break down the time specifications in both the faculty facts and class facts. This is accomplished by comparing the time facts with both the faculty and class facts. For example, assume a class fact indicates that a class may be taught Mondays and Wednesdays if the class starts at or after 1:00 p.m. and ends before 5:30 p.m.:

(class [number_assigned_to_class] CLASS_NUMBER:
[course number] CLASS_SEMESTER: CLASS_TIME:
M 1300 1730 T 0 0 W 1300 1730 TH 0 0 F 0 0 CLASS_ROOM:
CLASS_SECTIONS: [section]
CLASS_UNITS: [number_of_units] CLASS_WEIGHT:)

In this case, after comparing all the possible time facts to the class facts specified time, the following time facts match:

(time-chart 9 M 1300 1430 T 2400 0 W 1300 1430 TH 2400 0 F 2400 0)
(time-chart 10 M 1430 1600 T 2400 0 W 1430 1600 TH 2400 0 F 2400 0)
(time-chart 11 M 1600 1730 T 2400 0 W 1600 1730 TH 2400 0 F 2400 0)

The class fact is then stripped of any previous time designators and assigned time slots 9, 10 and 11:

(class [number_assigned_to_class] CLASS_NUMBER:
[course_number] CLASS_SEMESTER: CLASS_TIME:
9 10 11 CLASS_ROOM: CLASS_SECTIONS: [section]
CLASS_UNITS: [number_of_units] CLASS_WEIGHT:)

After all the class facts and faculty facts have gone through this operation, then the time facts are permanently deleted from the working memory.

Therefore, at the completion of Module 3, the class and faculty facts have new time designators and all the time facts are eliminated, which will free up memory for other operations.

2.2.4. Module 4. Module 4 is responsible for the initial assignment of classes to the faculty members. Starting with the highest priority Professor and ending with the lowest priority Professor, each faculty member attempts to obtain a single class of his or her choice. After all faculty members have had an opportunity to obtain a class, then the cycle will repeat.

There are many variables that are taken into account during this process. When a faculty member requests a specified class, a number of scenarios may occur. For instance, if the class does not exist, then the same faculty member will attempt to select another one of his or her preferred classes. In other words, the current faculty member may exhaust all of his or her class options until he or she obtains a single class during this cycle. For the sake of argument, assume a faculty member finds a class that he or she would prefer to teach. Next, the expert system verifies that the available class time slots match with one of the current faculty member's time slots. If it does, then the Professor obtains the class, the class is stricken from the available class list, the Professor looses the matched available time slot and the process begins for the next Professor in the cycle. However, if the time slots do not match, then the expert system looks for other possible sections of the class that may offer a more compatible time scenario. If no other sections are found, then the current Professor will request another preferred class.

A simplified rule representation of Module 4 may look something like this:

```
if (class_available) and (time_available)
    then (assign_class)
if (class_not_available) or (time_not_available)
    then (try_different_class_preference)
                    or
    then (try_different_class_section)
```

Table 2. Time block distribution for both classes and faculty

Days	Hours		
	Morning	Afternoon	Evening
Mon, Wed and Fri	0800–1300	—	—
Mon and Wed	—	1300–1730	1730–2200
Tue and Thur	0800–1230	1230–1730	1730–2200

Table 3. Assigned points for faculty time blocks

Days	Hours		
	Morning	Afternoon	Evening
Mon, Wed and Fri	2 points	—	—
Mon and Wed	—	1 point	1 point
Tue and Thur	1 point	1 point	1 point

```
if (no_more_class_preferences_for_current_Professor)
    then (go_to_next_Professor_in_cycle)
if (no_more_class_preferences_for_all_faculty_members)
    then (end_Module_4)
```

Although the above rules are general in form, the applied logic is similar to that found in Module 4. The end result of Module 4 is to assign as many of the requested courses as possible to the faculty and to record the number of units each faculty member has received.

Time blocks—To help guarantee the success of the expert system logic for Module 4, certain restrictions should be placed on both the class and faculty time availability. For class time availability, the classes should be available in at least one time block. The time blocks are designated as morning, afternoon or evening, as shown in Table 2.

Additionally, each full-time faculty member must obtain a total of at least three points for time availability. The points, as assigned in Table 3, are distributed according to the class load that may be obtained during the designated time block. Please note, the class scheduling expert system is capable of accepting class times or faculty times other than specified by the time blocks. The time blocks are used as recommendations to help guarantee that the faculty member will achieve his requested class load.

The use of time blocks will prevent the need to shift a Professor's class from one time slot to another. For example, assume time blocks are not used and a Professor obtains a class at a specified time slot. The same Professor then requests the addition of another course. If the requested course is at the same time as the previously obtained course and no other available times match, then the previously obtained course must be moved to a different time slot to allow the Professor to obtain both of these courses. This process may become quite complex, however, this technique is elminated by incorporating time blocks. Time blocks will help reduce the possibility of two classes having conflicting time slots. To follow up on the previous example, assume the Professor, whose available time is allocated in the time blocks found in Table 2, obtained a course that belonged in the same time block. In the example, the Professor's next requested class also belongs in the same time block as the previously obtained course. Since the time blocks can hold at least three courses, two time slots are available for the requested course without ever shifting the time slot of the previously obtained course.

2.2.5. Module 5. Module 5 transforms the results from Module 4 into more suitable fact representations for Module 6. In Module 4 each Professor has obtained a given set of classes. The first class obtained by the faculty member will be his or her most preferred course. Conversely, the last class obtained by the faculty member will be his or her least preferred course. Module 5 sets up the courses accordingly, as well as recording the respective section number and class time slot, for each faculty member. Along with this, Module 5 will locate the faculty member with the largest collection of obtained classes and then manipulate all the other faculty members to have the same number of classes. For example, if a faculty member with the most classes had a total of five courses, then all other faculty members will also have five courses. Therefore, if a Professor only obtained two courses, he or she would then be assigned three additional courses, all with a course number of 0. The reasoning for this process will be demonstrated in logic Module 6.

Requested course priorities—Portions of logic Module 5 are responsible for sorting the obtained classes according to preference. As shown in Table 4, the most preferred class will be located in the leftmost portion of the faculty member's fact. Conversely, if the faculty member expresses little concern for obtaining a particular course, then that course will be placed in the right most portion of that faculty members fact. This structure will supply Module 6 with the knowledge of which courses are important or not important to a Professor's personal needs.

How are class preferences obtained? Initially, the course selection knowledge is obtained from each individual faculty member. The Professor is asked the following:

Which classes would you prefer to teach?
Which classes would you consider teaching?
Which classes, which are not answers to the previous questions, have you recently taught?
Which classes are in the Professor's emphasis?

The response to the first two questions would yield knowledge concerning the faculty members' course needs while the third question provides information on the Professor's capabilities. The last question signifies the courses in which the faculty member did not request or previously teach, but might have a strong enough background or interest in the material to do so. The class scheduling expert system will combine the knowledge into one serial data string. The data string will begin with the preferred courses which would be followed by classes that the faculty member would consider teaching and then end with courses that the Professor has previously taught or courses in the Professor's emphasis curriculum. Therefore, the leftmost portion of the data string are the most preferred courses while the right-hand side of the data string are the least preferred.

2.2.6. Module 6. In Module 4 faculty members were assigned, if possible, their requested courses. Unfortunately, even though some faculty members were assigned more classes than they require, not all faculty members are likely to have obtained their requested course-load. The objective of Module 6 is to logically deduce class transactions between the faculty members that have too many units and the faculty members that have a deficient number of units.

Initially, Module 6 will label the faculty members with too few units or the minimal requested units as "off limits". These Professors cannot afford to give any courses to any faculty members that might need them. The expert system will constantly update the "off limits" list after each class transaction between faculty members. For example, assume a faculty member requested 12 units but had obtained 15 units. If the faculty member gave a three unit course to another faculty member, then he or she would be left with the minimum requested units. To prevent any more loss of units from this faculty member, he or she must be considered "off limits" before another course transaction occurs.

Selecting the faculty members for a course transaction is *not* a simple random process. If that were the case, then the Professors with the greater seniority would unfairly yield their opportunity to obtain preferred courses to faculty members who have not yet earned the higher priority rank. Also, the random class transaction technique could possibly remove a highly preferred class instead of a less preferred class from a faculty member.

Module 6 contains the logic to make fair decisions on which Professor is to yield what course to which other Professor. Assume the logic in Module 5 created the following scenario:

Table 4. Class transaction example

Faculty Member	Rank	Obtained courses		
		1st Choice	2nd Choice	3rd Choice
Professor A	1	X1	X2	X3
Professor B	2	Y1	Y2	0
Professor C	3	Z1	Z2	Z3

Module 6 will first locate the faculty member who has the highest priority and needs additional units to accommodate his requested course load. Assume that Professor D, not shown, meets these requirements. Also, assume the Professors in Table 4 all have at least one unneeded course, even though Professor B has only obtained two classes. In Module 6, Professor D will compare all his or her needed but unobtainable courses with the least preferred course from the Professor who has the most courses and the lowest priority. In this case, Professor D will investigate course Z3 from Professor C. If the course is not needed by Professor D or if the course is needed but provides incompatible time slots, then Module 6 will proceed to give Professor D the opportunity to investigate Professor B's obtained courses. Unfortunately for Professor D, Professor B has obtained less courses than Professor A and Professor C, therefore Professor B's courses are not

yet examined. To complete the first cycle, Professor D will compare his or her requested courses with Professor A's X3 course. If unsuccessful, then a new cycle will begin and Professor D will inspect Professor C's next preferred course, class Z2. Next, Professor D will compare his or her requested courses with class Y2 from Professor B and then class X2 from Professor A. At the end of this cycle, the next cycle begins with the sequential comparison of the first-class choices of Professor C, Professor B and then Professor A. In any of the above cycles, if Professor D locates the needed course with an acceptable available time slot, then Professor D will obtain the course and the Professor who yielded the course will become "off-limits". Overall, logic Module 6 will continue to operate until all faculty members have obtained the necessary class loads or until no more classes can be traded.

There are quite a few details that must be covered by Module 6. For instance, when a match of classes does occur, the expert system will go back to the class information generated by Module 3 for a complete analysis of available class time slots. Other details include retaining and updating facts on the faculty member's obtained courses, requested courses, and available time slots. Please keep in mind that every time a Professor gains a course, he loses a time slot, and vice versa. A generic logical rule outline for module 6 may look something like this:

```
if (beginning_of_module_6)
   then (compare_class_of_faculty_member_
         with_the_most_classes_and_the_
         least_priority)

if (not_end_of_cycle) and (no_class_match)
   then (compare_the_same_choice_level_class_
         from_the_next_higher_priority_level_
         faculty_member)

if (end_of_cycle) and (no_class_match)
   then (begin_new_cycle_at_the_next_lower_
   class_choice)

if (end_of_cycle) and (end_of_choices)
   then (find_next_lower_priority_faculty
   member_who_needs_a_course)

if (end_of_cycle) and (end_of_choices_for_faculty)
   then (end_module_6)

if (class_match) and (appropriate_time_slot)
   then (class_transaction)

if (class_match) and (not_appropriate_time_slot)
   then (continue_cycle)
```

As previously stated, the above rules are merely generic representations and it should not be construed that these rules encompass all the situations that module 6 is capable of dealing with.

2.2.7. Module 7. At this phase of the process, most class scheduling decisions have been completed. However, for the user, the solutions appear to be partially indecipherable. Logic Module 7 will return the results to a more user-friendly form.

The first portion of Module 7 is to break the facts into manageable knowledge. This may include removal of comments attached to each fact by the expert system. Next, all unneeded classes will be removed from each faculty member's obtained course list. For example, if a faculty member requests 12 units, but obtains six classes of three units each, then Module 7 will remove the two least-preferred courses. And finally, the results are displayed on the CRT and are stored in the previously mentioned Result File, for permanent reference. Please note that similar techniques are used to store data to a file as were required to read data from a file in logic Module 2.

3. EXAMPLE

Using the approach presented in this paper, a test sample of 11 faculty members and a selection of 40 courses was created to extensively test the capabilities of the class scheduling expert system.

As shown by the following list of the 11 faculty members and their inputs, complicated situations were added to further test the class scheduling expert system. For example, not all of the faculty members adhered to the time block standard, one faculty member is on sabbatical and another faculty member requested an unusual class load which cannot be evenly divided by the typical 3 unit course.

Faculty Input File

PROF_PRIORITY: 1
PROF_NAME: Armstrong
PROF_CHOICE: 314 403 519 416 425
PROF_TIME: M 800 1730 T 0 0 W 800 1730 TH 0 0 F 800 1300
PROF_UNITS: 11

PROF_PRIORITY: 2
PROF_NAME: Boole
PROF_CHOICE: 201 404 527 519 311 582
PROF_TIME: M 0 0 T 800 1730 W 0 0 TH 800 1730 F 0 0
PROF_UNITS: 15

PROF_PRIORITY: 3
PROF_NAME: Curie
PROF_CHOICE: 201 203 307 412 455
PROF_TIME: M 800 1730 T 0 0 W 800 1730 TH 0 0 F 800 1300
PROF_UNITS: 12

PROF_PRIORITY: 4
PROF_NAME: Doppler
PROF_CHOICE: 205 420 558 309 313
PROF_TIME: M 1300 1730 T 1730 2200 W 1300 1730 TH 2200 F 0 0
PROF_UNITS: 12

PROF_PRIORITY: 5
PROF_NAME: Einstein
PROF_CHOICE: 302 203 448 580 420
PROF_TIME: M 0 0 T 1230 2200 W 0 0 TH 1230 2200 F 0 0
PROF_UNITS: 12

PROF_PRIORITY: 6
PROF_NAME: Faraday
PROF_CHOICE: 304 425 205 310 445 412
PROF_TIME: M 0 0 T 800 1730 W 0 0 TH 800 1730 F 0 0
PROF_UNITS: 9

PROF_PRIORITY: 7
PROF_NAME: Gauss
PROF_CHOICE: 304 308 442 527 489
PROF_TIME: M 0 0 T 800 1730 W 0 0 TH 800 1730 F 0 0
PROF_UNITS: 15

PROF_PRIORITY: 8
PROF_NAME: Hertz
PROF_CHOICE:
PROF_TIME: M 0 0 T 0 0 W 0 0 TH 0 0 F 0 0
PROF_UNITS: 0

PROF_PRIORITY: 9
PROF_NAME: Jacob
PROF_CHOICE: 307 442 448 416 308 580
PROF_TIME: M 800 1730 T 0 0 W 800 1730 TH 0 0 F 800 1300
PROF_UNITS: 9

PROF_PRIORITY: 10
PROF_NAME: Karnaugh
PROF_CHOICE: 304 404 455 424 310
PROF_TIME: M 1730 2200 T 800 1230 W 1730 2200 TH 800 1230 F 0 0
PROF_UNITS: 12

PROF_PRIORITY: 11
PROF_NAME: Laplace
PROF_CHOICE: 308 442 558 416 425
PROF_TIME: M 1730 2200 T 1230 1600 W 1730 2200 TH 1230 1600 F 0 0
PROF_UNITS: 9

The following is a partial list of the courses in which the faculty members could make their selection from.

Class Input File (partial list):

CLASS_NUMBER: 314
CLASS_SEMESTER:
CLASS_TIME: M 800 1300 T 0 0 W 800 1300 TH 0 0 F 800 1300
CLASS_ROOM:
CLASS_SECTIONS: 1
CLASS_UNITS: 2
CLASS_WEIGHT: 2

CLASS_NUMBER: 201
CLASS_SEMESTER:
CLASS_TIME: M 0 0 T 1230 1730 W 0 0 TH 1230 1730 F 0 0
CLASS_ROOM:
CLASS_SECTIONS: 1
CLASS_UNITS: 3
CLASS_WEIGHT: 3

CLASS_NUMBER: 203
CLASS_SEMESTER:
CLASS_TIME: M 1300 1730 T 0 0 W 1300 1730 TH 0 0 F 0 0
CLASS_ROOM:
CLASS_SECTIONS: 1
CLASS_UNITS: 3
CLASS_WEIGHT: 3

CLASS_NUMBER: 205
CLASS_SEMESTER:
CLASS_TIME: M 1300 1730 T 0 0 W 1300 1730 TH 0 0 F 0 0
CLASS_ROOM:
CLASS_SECTIONS: 1
CLASS_UNITS: 3
CLASS_WEIGHT: 3

CLASS_NUMBER: 302
CLASS_SEMESTER:
CLASS_TIME: M 0 0 T 1730 2200 W 0 0 TH 1730 2200 F 0 0
CLASS_ROOM:
CLASS_SECTIONS: 1
CLASS_UNITS: 3
CLASS_WEIGHT: 3

CLASS_NUMBER: 304
CLASS_SEMESTER:
CLASS_TIME: M 0 0 T 800 1230 W 0 0 TH 800 1230 F 0 0
CLASS_ROOM:
CLASS_SECTIONS: 2

CLASS_UNITS: 3
CLASS_WEIGHT: 3
CLASS_NUMBER: 308
CLASS_SEMESTER:
CLASS_TIME: M 0 0 T 1230 1730 W 0 0 TH 1230 1730 F 0 0
CLASS_ROOM:
CLASS_SECTIONS: 1
CLASS_UNITS: 3
CLASS_WEIGHT: 3

The class scheduling expert system proved to be completely successful. At the conclusion of logic Module 4, only 40% of the faculty members had received their requested coarse loads, however, at the end of logic Module 7 all the faculty members had acquired their requested course loads as indicated by the following Result File.

Result File:

Professor Armstrong, Priority: 1
class: 416 section: 1 time: MWF 1100–1150
class: 314 section: 1 time: MWF 800–850
class: 403 section: 2 time: MWF 900–950
class: 519 section: 1 time: MW 1300–1415

Professor Boole, Priority: 2
class: 201 section: 1 time: TTH 1230–1345
class: 404 section: 1 time: TTH 1400–1515
class: 519 section: 2 time: TTH 930–1045
class: 311 section: 1 time: TTH 1100–1215
class: 582 section: 1 time: TTH 1600–1715

Professor Curie, Priority: 3
class: 307 section: 1 time: MWF 1000–1050
class: 203 section: 1 time: MW 1300–1415
class: 412 section: 1 time: MW 1430–1545
class: 455 section: 2 time: MWF 800–850

Professor Doppler, Priority: 4
class: 205 section: 1 time: MW 1300–1415
class: 558 section: 1 time: MW 1430–1545
class: 309 section: 1 time: TTH 1900–2015
class: 313 section: 1 time: MW 1600–1715

Professor Einstein, Priority: 5
class: 420 section: 1 time: TTH 2030–2145
class: 302 section: 1 time: TTH 1730–1845
class: 203 section: 2 time: TTH 1230–1345
class: 580 section: 1 time: TTH 1900–2015

Professor Faraday, Priority: 6
class: 205 section: 2 time: TTH 930–1045
class: 445 section: 1 time: TTH 1400–1515
class: 412 section: 2 time: TTH 1600–1715

Professor Gauss, Priority: 7
class: 304 section: 2 time: TTH 930–1045
class: 527 section: 1 time: TTH 1100–1215
class: 508 section: 1 time: TTH 1230–1345
class: 442 section: 1 time: TTH 1400–1515
class: 489 section: 1 time: TTH 800–915

Professor Hertz, Priority: 8

Professor Jacob, Priority: 9
class: 448 section: 1 time: MWF 900–950
class: 308 section: 2 time: MW 1300–1415
class: 580 section: 1 time: MW 1900–2015

Professor Karnaugh, Priority: 10
class: 310 section: 1 time: TTH 1100–1215
class: 304 section: 1 time: MW 1730–1845
class: 455 section: 1 time: TTH 800–915
class: 424 section: 1 time: MW 1900–2015

Professor Laplace, Priority: 11
class: 425 section: 1 time: TTH 1400–1515
class: 308 section: 3 time: MW 1730–1845
class: 442 section: 2 time: MW 1900–2015

Please note that all faculty members from the above list acquired the same number of courses, this is due to the fact that not all of the Professors requested the same course load.

4. SUMMARY

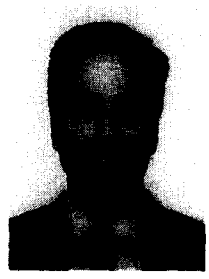
A rule-based expert system for class scheduling was successfully developed and implemented. The human decision-making process required to perform a class scheduling task was divided into its basic elements. The reasoning derived from these elements was then reproduced by a set of rules contained in each of the logic modules. The logic modules were then combined with a user friendly interface to effectively and reliably emulate the class scheduling process [9].

Please note that although a PC-based platform was chosen to perform the expert system evaluation, the batch program was successfully designed to avoid exceeding the computer memory limits by assigning only one logic module to memory at a time.

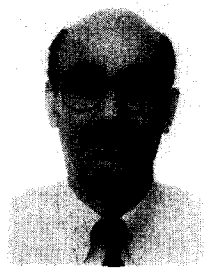
REFERENCES

1. M. Fox, AI and expert system myths, legends and facts. *IEEE Expert* **Feb,** 8–20 (1990).
2. J. Giarratano and G. Riley, *Expert Systems: Principles and Programming.* PWS-KENT, Boston, MA (1989).
3. C. W. Butler and E. D. Hodil, Building knowledge-based systems with procedural languages. *IEEE Expert* **Summer,** 47–59 (1988).
4. P. E. Slatter, *Building Expert Systems: Cognitive Emulation.* Halsted Press, New York (1987).
5. J. Mason, Putting what a human expert knows into the computer. *PC Week* **28 July,** 59 (1987).
6. M. Stoll, PC-based expert-system shells are bringing AI to mainstream America *PC Week* **June 2** (1987).
7. C. Williams, Expert systems, knowledge engineering, and AI tools—an overview. *IEEE Expert,* **Winter,** 66–70 (1986).
8. M. Schor, Declarative knowledge programming: better than procedural. *IEEE Expert,* **Spring,** 36–43 (1986).
9. L. Guyette, *Class Scheduling Expert System.* MS Thesis, California State University, Fullerton (1991).

AUTHORS' BIOGRAPHIES



Lee G. Guyette—Lee G. Guyette was born in Alexandria, Virginia. He received a B.S. and M.S. in Electrical Engineering from California State University, Fullerton in 1986 and 1991 respectively. Mr Guyette is a senior electrical engineer at the Naval Warfare Assessment Center. He has 7 yr of engineering and software development experience, which includes his current work with the integration of artificial intelligence into a newly developed Fleet exercise reconstruction and debrief system. Previously, he worked as a software engineer for General Dynamics.



Karim Hamidian—Karim Hamidian was born in Maragheh, Iran. He received his doctorate degree in Electronics Engineering from the University of Padova, Italy in 1981. He has provided consulting service for several years to Omegatech Co., Gas Power Systems Int., General Electronics Co. and Management and Industrial Engineers. Dr Hamidian joined the Electrical Engineering Department of California State University, Fullerton in 1984 as a lecturer. Presently he is an associate professor of Electrical Engineering. In 1992 he was appointed as Acting Associate Dean for Academic Affairs for the School of Engineering and Computer Science. His research interests include communications theory, information theory, neural networks and their applications with emphasis on spread spectrum and multi-user communication systems.

Jesus O. Tuazon—Jesus O. Tuazon received his M.S. and Ph.D. in Electrical Engineering at Iowa State University, Ames, Iowa in 1966 and 1969 respectively. He has been a professor in Electrical Engineering at California State University, Fullerton since 1969. He is also a consultant to the Jet Propulsion Laboratory of California Institute of Technology, Pasadena, California. His research area of specialties are in parallel processing, fault-tolerant system and neural networks. He was involved in the Hypercube project at JPL.